

“CommunityLearn: Web-Based Student–Mentor Platform with Secure Authentication”

An Engineering Project in Community Service

Phase – II Report

Submitted by

Sl. No.	Register Number	Name
1	23BCE10309	PRATHAM MITTAL
2	23BAI10872	SAUMIL SINGHAL
3	23BSA10035	SHIKHAR GUPTA
4	23BCY10191	PRAVEEN SINGH
5	23BCY10119	AMAN KUMAR GUPTA
6	23BCE10380	SAMBHAV CHHAYALA

in partial fulfillment of the requirements for the degree of
Bachelor of Technology



VIT Bhopal University
Bhopal
Madhya Pradesh

March,2026



Bonafide Certificate

Certified that this project report titled “**CommunityLearn: Web-Based Student–Mentor Platform with Secure Authentication**” is the bonafide work of “**PRATHAM MITTAL (23BCE10309) , SAUMIL SINGHAL (23BAI10872) , SHIKHAR GUPTA (23BSA10035) , PRAVEEN SINGH (23BCY10191) , AMAN KUMAR GUPTA (23BCY10119) , SAMBHAV CHHAYALA (23BCE10380)**” who carried out the project work under my supervision.

This project report (Phase II) is submitted for the Project Viva-Voce examination held on ...31/03/26.....

Supervisor

Comments & Signature (Reviewer 1)

Comments & Signature (Reviewer 2)



Declaration of Originality

We, hereby declare that this report entitled **“CommunityLearn: Web-Based Student–Mentor Platform with Secure Authentication”** represents our original work carried out for the EPICS project as a student of VIT Bhopal University and, to the best of our knowledge, it contains no material previously published or written by another person, nor any material presented for the award of any other degree or diploma of VIT Bhopal University or any other institution. Works of other authors cited in this report have been duly acknowledged under the section "References".

Date:31/03/26

Reg No & Name

23BCE10309- PRATHAM MITTAL

23BAI10872- SAUMIL SINGHAL

23BSA10035- SHIKHAR GUPTA

23BCY10191- PRAVEEN SINGH

23BCY10119- AMAN KUMAR GUPTA

23BCE10380- SAMBHAV CHHAYALA

Acknowledgement

We express our sincere gratitude to VIT Bhopal University for providing the infrastructure and academic environment for this Engineering Project in Community Service (EPICS). We thank our project supervisor for constant guidance, constructive feedback, and motivation throughout Phase II. We are also thankful to faculty members of the School of Computing and Engineering for their valuable suggestions. Finally, we thank our peers, community learners, friends, and families for their continuous support.

Abstract

CommunityLearn is an e-mentoring platform developed for facilitating interaction and communication between students and mentors through an online interface. The system has been designed to eliminate major drawbacks of traditional mentoring techniques, including poor organization of communication sessions and lack of central management for tracking sessions. Moreover, there is often poor implementation of secure user authentication and password recovery.

Role-based access has been integrated into the system for facilitating interaction and communication between mentors and students. Users can register and log in to the system based on their roles. The system is designed for easy and smooth interaction through its frontend interface.

A major aspect of the system is its focus on security and reliability. JSON Web Token (JWT)-based authentication has been implemented for facilitating secure sessions and restricting access to authorized users only. Moreover, an OTP-based password recovery system has been integrated into the system for facilitating password recovery through NodeMailer. This ensures high levels of account security and provides users with a smooth password recovery experience.

The system is developed based on a full-stack architecture that incorporates React for the front-end development, Node.js and Express for the back-end development, and PostgreSQL for the database management system. The back-end development incorporates features such as authentication, sessions, API routing, and database management. On the other hand, the front-end development incorporates features that ensure an intuitive interface for interaction.

This Phase II system development represents the transition from conceptual design to actual working system development. The developed system has successfully validated the proposed system architecture and its application in real-world educational environments.

Overall, CommunityLearn is an effective system that improves accessibility, security, and efficiency in mentoring systems through interaction and management. The system provides a strong basis for further development to include real-time communication features and improve user experience.

Index

Sl. No.	Topic	Page No.
1.	Introduction	9
2.	Motivation	10
3.	Objective	11
4.	Existing Work / Literature Review	12
5.	Topic of the Work	15
6.	System Design / Architecture	16
7.	Working Principle	19
8.	Results and Discussion	22
9.	Individual Contribution by Members	23
10.	Conclusion	26
11.	References	27
12.	Publication/ Conference / Patent	29
12.	Biodata	30

List of Figures

Sl.No.	Caption	Page.no
1	High-Level architecture of CommunityLearn	18
2	Workflow of User Interaction and Authentication	20

List of Tables

Sl.No	Caption	Page.No
1	Phase II Implementation Milestones	22
2	Achieved Outcomes of Phase II	22

INTRODUCTION

Community mentoring has also played an important role in the support of students. Traditional mentoring systems have often taken place in informal environments, and the level of communication between the mentors and the students has not been well structured. This has led to challenges like the inability to track the students and the mentors, the level of interaction between the mentors and the students, and the inability to manage the students. This has had a great effect on the learning experience of the students.

The use of technology has also improved in the world of learning, and this has led to the need to develop the mentoring systems. This has led to the need to develop the mentoring systems in the modern world. A web-based mentoring system has the ability to bridge the gap between the students and the mentors.

CommunityLearn is developed as a web-based e-mentoring platform that provides a structured environment for students and mentors to interact effectively. This platform has the ability to support role-based access, and users are able to register and log in as students and mentors. Each user is given a personal dashboard that helps them to manage all their activities.

One of the major features of this system is secure user authentication. JSON Web Token (JWT) is used for authenticating users and maintaining sessions. Furthermore, an OTP-based password recovery system is incorporated through email services to allow users to recover their passwords in case of forgotten credentials. This makes the system more reliable and secure.

A full-stack development approach is used to design this system. For instance, React is used to create the frontend of the system to build an interactive interface for users. Node.js and Express are used for building the backend of the system to handle all application-level requests and store data in a PostgreSQL database. The system maintains smooth communication between its frontend and backend through well-defined APIs.

This phase, being the implementation phase of the project, ensures the conversion of the conceptual design into an operational system. The project, therefore, proves the successful implementation of key features such as user authentication, role-based dashboards, session management, and password recovery. The developed system, therefore, represents an operational solution for the enhancement of the mentoring process within an educational setting.

In summary, the project, known as CommunityLearn, seeks to improve the mentoring process through the development of a secure, accessible, and well-structured online platform

MOTIVATION

CommunityLearn is inspired by the challenges that are experienced in real-world situations, especially during interactions with community-based tutoring spaces and learning environments. In most of these places, mentoring is done without a proper system for maintaining records of the students, tracking the progress of the students, or ensuring proper communication between the mentors and the students. The mentors are required to cater to the needs of multiple students, which makes it difficult to properly organize the mentoring process and provide timely mentoring to the students. This affects the students adversely, as the mentoring process is not continuous, and the students are not guided properly in their learning journey.

In the current state, the students are confused about what to study next, how to improve their grasp of certain topics, and what resources to use to improve their learning. Without a proper system, the mentors and the students are forced to rely on a disorganized process, which affects the mentoring process adversely, especially in terms of the time taken to provide the required mentoring to the students. The mentors are also unable to track the progress of the individual students, as they lack the proper tools to do the same.

Another major concern is the lack of proper and safe authentication for users, especially for small-scale or unorganized systems, as users might experience difficulties like forgotten passwords, insecure login procedures, or lack of proper password recovery mechanisms. This might lead to a poor user experience for the users.

The CommunityLearn project resolves the above-mentioned problems by offering a safe, web-based mentoring platform that organizes the interactions between the students and mentors in a proper manner, thereby enhancing the usability and efficiency of the mentoring platform for the users. The project also incorporates a safe authentication procedure for the users, employing JWT, as well as a password recovery procedure employing OTP for the users, thereby ensuring the safety of the users' accounts.

The above-mentioned improvements to the mentoring platform by the CommunityLearn project make the mentoring experience more efficient, safe, and appropriate for the current era of education.

OBJECTIVE

The CommunityLearn project aims to design and develop a web-based e-mentoring platform that facilitates structured interaction between students and mentors. The primary objective of this system is to create a centralized digital environment where users can register, log in, and access role-based functionalities through a secure and user-friendly interface. The platform focuses on improving communication, organization, and accessibility in mentoring systems.

One of the key objectives is to implement a secure authentication mechanism to protect user data and ensure safe access to the platform. This includes the use of JSON Web Token (JWT)-based authentication for session management and an OTP-based password recovery system using email services. These features are intended to enhance security and provide a reliable solution for account management.

Another important objective is to develop role-based dashboards for students and mentors, allowing each user to access relevant features according to their role. Mentors can manage sessions, view student information, and monitor interactions, while students can access mentoring support, view session details, and interact within the platform. This structured approach helps in improving efficiency and usability.

The CommunityLearn project seeks to design and develop a web-based e-mentoring platform that enables the interactive process between students and mentors to be more structured. The major goal of the e-mentoring platform is to provide a digital space for users to register, login, and access role-based functionality via a safe, efficient, and user-friendly interface. The e-mentoring platform is focused on improving the interactive, organizational, and accessible nature of the mentoring system.

One of the major objectives of the project is to provide a safe authentication system for users to access the e-mentoring platform while protecting the users' information from unauthorized access. This can be done by implementing a JSON Web Token (JWT)-based authentication system for the session management process and a One-Time Password (OTP)-based password recovery system via email services.

The other major objective of the project is to design and implement role-based functionality for the students and mentors, which enables users to access the functionality of the e-mentoring platform according to their roles. The mentors can access the mentoring session, the information of the students, and the interactive session, while the students can access the mentoring support, the session information, and the interactive session via the e-mentoring platform.

EXISTING WORK / LITERATURE REVIEW

Over the past decade, web-based educational platforms have gained significant attention as a means to improve accessibility, organization, and effectiveness of learning systems. Various research efforts have focused on developing e-learning platforms, mentoring systems, and secure web applications that enhance interaction between users and streamline academic processes. However, most existing systems are either limited in functionality or lack proper integration of essential features such as secure authentication, structured dashboards, and efficient communication mechanisms.

1. Web-Based Mentoring Systems

Web-based mentoring systems are designed to provide structured communication between mentors and learners through online platforms. These systems aim to replicate traditional mentoring relationships in a digital environment while improving accessibility and efficiency. Early mentoring systems primarily focused on basic communication features such as messaging and simple profile management. However, they lacked proper organization, tracking mechanisms, and scalability.

With the evolution of web technologies, modern mentoring platforms have introduced features such as session scheduling, progress tracking, and user management. These systems allow mentors to manage multiple students simultaneously and provide guidance more effectively. However, many existing systems are either too complex or not optimized for real-world usage in community-based environments.

A significant limitation observed in existing mentoring systems is the lack of centralized management. In many cases, communication between mentors and students is scattered across different tools such as emails, messaging platforms, or manual records. This fragmentation leads to inefficiencies and makes it difficult to track student progress over time. Additionally, many systems do not provide role-based access, which is essential for differentiating functionalities between mentors and students.

Community-based mentoring environments present additional challenges, including irregular schedules, diverse student needs, and limited technological infrastructure. Most existing systems are designed for formal educational institutions and may not be suitable for flexible, community-driven learning setups. This highlights the need for a simplified and structured platform that can effectively support mentoring activities in such environments.

2. E-Learning Platforms

E-learning platforms have become an integral part of modern education, providing learners with access to educational resources, online courses, and virtual classrooms. These platforms enable users to learn at their own pace and access content from anywhere, making education more accessible and flexible.

Research in this area has focused on improving user experience, content delivery, and system scalability. Many platforms incorporate features such as multimedia content, assessments, and discussion forums to enhance learning outcomes. However, most e-learning systems primarily focus on content delivery rather than interactive mentoring.

One of the key limitations of existing e-learning platforms is the lack of personalized

interaction between learners and instructors. While these systems provide access to a vast amount of information, they often fail to address individual learning needs. In community-based learning environments, where personalized guidance is essential, this limitation becomes more significant.

Additionally, many e-learning platforms are designed for large-scale deployment in universities or corporate training environments. These systems may require complex infrastructure and may not be suitable for smaller, community-driven setups. The absence of structured mentoring features further limits their effectiveness in supporting collaborative learning.

3. Authentication and Security in Web Applications

Security is a fundamental aspect of modern web applications, particularly in systems that handle sensitive user data. Authentication mechanisms play a crucial role in ensuring that only authorized users can access system resources. Over the years, various authentication methods have been developed, including session-based authentication, token-based authentication, and multi-factor authentication.

JSON Web Token (JWT) has become a widely adopted standard for secure authentication in web applications. JWT enables stateless authentication, allowing servers to verify user identity without maintaining session data. This approach improves scalability and reduces server overhead. It is particularly useful in full-stack applications where frontend and backend components communicate through APIs.

In addition to authentication, password recovery mechanisms are essential for enhancing user experience and system reliability. OTP-based password recovery systems have gained popularity due to their simplicity and effectiveness. By sending a one-time password to the user's registered email, these systems provide a secure method for verifying user identity during password reset processes.

Despite the availability of these technologies, many educational platforms still lack robust security mechanisms. Weak authentication systems can lead to unauthorized access, data breaches, and poor user trust. Integrating secure authentication with ease of use remains a key challenge in system design.

4. Dashboard-Based Systems

Dashboard-based interfaces have become a standard feature in modern web applications, providing users with a centralized view of relevant information and functionalities. Dashboards improve usability by organizing data and features in a structured and intuitive manner.

In educational systems, dashboards play a crucial role in enhancing user experience. Mentors can use dashboards to manage sessions, view student details, and monitor interactions, while students can access their learning activities and track progress. Role-based dashboards further improve system efficiency by providing customized views for different types of users.

However, many existing systems either lack proper dashboard design or provide overly complex interfaces that are difficult to navigate. Poorly designed dashboards can reduce usability and hinder user engagement. Therefore, creating simple, intuitive, and role-

specific dashboards is essential for improving system effectiveness.

5. Full-Stack Web Application Development

The development of full-stack web applications has become increasingly popular due to the need for integrated and scalable systems. Modern applications typically consist of a frontend for user interaction, a backend for handling logic and data processing, and a database for storing information.

Technologies such as React, Node.js, and Express have gained widespread adoption in full-stack development. React enables the creation of dynamic and responsive user interfaces, while Node.js and Express provide a robust backend framework for handling API requests and server-side logic. PostgreSQL is commonly used as a relational database for managing structured data.

Research in this area emphasizes the importance of modular architecture, efficient API design, and seamless communication between frontend and backend components. However, implementing a well-integrated system that combines all these aspects remains a challenge, particularly in educational applications.

6. Gaps in Existing Systems

Based on the analysis of existing literature, several gaps can be identified in current web-based mentoring and learning systems:

1. Lack of integrated platforms that combine mentoring, authentication, and session management.
2. Limited focus on community-based learning environments with flexible structures.
3. Inadequate implementation of secure authentication and password recovery mechanisms.
4. Absence of well-designed role-based dashboards for different users.
5. Challenges in scalability and efficient data management.
6. Over-reliance on complex systems that are not suitable for small-scale implementations.

7. Summary

The review of existing work indicates that while significant progress has been made in web-based education and mentoring systems, there is still a need for a unified, secure, and user-friendly platform that addresses the specific requirements of community-based learning environments.

CommunityLearn is developed to bridge these gaps by providing a full-stack web-based mentoring system that integrates secure authentication, role-based dashboards, and structured interaction between users. The system emphasizes simplicity, security, and scalability, making it suitable for real-world educational applications. By combining essential features into a single platform, CommunityLearn enhances the overall mentoring experience and provides a practical solution for modern learning environments.

TOPIC OF THE WORK

This paper discusses the design and implementation of CommunityLearn – a web-based mentoring and learning platform developed to improve structured interaction between students and mentors. The system is designed to provide a centralized digital environment where learners can register, securely access their accounts, explore resources, and interact with the platform, while mentors can manage their sessions and monitor student engagement.

CommunityLearn is designed as a modular system consisting of clearly defined components. The User Management module handles user registration, authentication, and role-based access control for students and mentors. It ensures secure login using modern authentication techniques and provides features such as OTP-based password recovery for enhanced security. The Dashboard module offers personalized interfaces for different users, allowing students to access learning materials and mentors to manage their activities efficiently.

The system also includes a Resource Management module that enables the organization and presentation of educational content. Students can explore curated learning materials, while mentors can structure and provide resources in a systematic way. Additionally, a Quiz and Interaction module is included to support basic academic engagement, allowing students to attempt quizzes and interact with different learning components within the platform.

A key feature introduced in CommunityLearn is the Competitive Exams Preparation section. This module provides students with curated external resources for exams such as GATE, CAT, JEE, and UPSC. It includes direct links to YouTube courses, PDF materials, and other learning platforms. The aim of this feature is to extend the platform beyond basic mentoring and provide practical value for students preparing for competitive examinations.

The system follows a client-server architecture where the frontend, developed using React, communicates with a backend server built using Node.js and Express. The backend handles API requests, business logic, and data processing, while PostgreSQL is used as the database for storing user information and system data securely. The integration between frontend and backend ensures smooth data flow and real-time interaction.

From a functional perspective, the system includes user registration and login, secure authentication, password reset using OTP verification, dashboard access, resource viewing, and basic quiz handling. These features form the core functionality of the platform and reflect the actual implementation of the project.

Non-functional requirements such as usability, performance, and security are also considered. The platform is designed to be responsive and accessible across different devices, including desktops and mobile devices. Security is maintained through proper validation, secure authentication mechanisms, and controlled access to system resources. The system is also designed to be scalable, allowing future enhancements and additional modules to be integrated easily.

System Design / Architecture

1. Introduction to System Architecture

CommunityLearn is built using a multi-layered architecture that is modular and scalable to provide reliability, maintainability, and efficient operation within a web-based mentoring environment. The architecture is based on key components such as Client Browser, Web Frontend, Backend API Server, and Database. Each component is responsible for handling individual functions, allowing for smooth interaction among users, the system, and data management within the system.

The multi-layered architecture provides better separation among components, allowing for better maintenance and scalability within a web environment. Unlike traditional systems that may require manual handling and coordination among components, CommunityLearn is based on a digital platform that provides better integration among user authentication, management, and mentoring within a single system.

The architecture is based on a real-world environment for educational needs, allowing users to access the system using different devices and network environments. By maintaining different components within the architecture, the system is more likely to enhance and integrate new features without affecting existing functions within the system. The Phase II development is based on delivering a fully functional system with key features such as user authentication, dashboard management, and communication.

2. Presentation Layer: Client Browser and Web Frontend

The Presentation Layer consists of the Client Browser and the Web Frontend. The Client Browser represents the interface through which users such as students and mentors access the system. This includes devices such as desktops, laptops, tablets, and mobile phones. The system is designed to be responsive so that users can interact with the platform smoothly across different devices.

The Web Frontend is responsible for displaying the user interface and handling user interactions. It is developed using modern web technologies including HTML, CSS, JavaScript, and React. The frontend provides role-based interfaces, allowing users to view different features based on whether they are students or mentors.

This layer manages functionalities such as login, registration, dashboard display, and password recovery workflows. It communicates with the Backend API Server through secure HTTP requests and ensures that user inputs are validated before being sent to the server. The frontend is designed to provide a clean and user-friendly experience, making it easy for users to navigate the platform.

3. Application Layer: Backend API Server

The Backend API Server is essentially the heart of the application as it is responsible for managing all business logic operations as well as interactions between the frontend and database. The Backend API Server is created using Node.js and Express.js. This gives it an advantage as it is able to provide an ideal environment in which to process all user requests.

The backend is essentially tasked with managing all operations in an application. This includes managing all API routes as well as processing all requests made by the frontend. Additionally, it is tasked with managing all operations related to user authentication. The backend uses JSON Web Token to provide secure authentication as it is able to guarantee that only authorized persons gain access to an application.

Some of the major operations of the Backend API Server include:

- User registration as well as login
- Generation of JWT token
- OTP generation as well as email-based password recovery
- User role management
- Making API requests
- User input validation

The backend uses a stateless architecture. This gives it an advantage as it is able to effectively cater to all needs of an application as it is able to process all requests made by various users.

4. Data Layer: Database Component

The Backend API Server is essentially the heart of the application since it is responsible for handling all business logic operations and all frontend and database interactions. The Backend API Server is created using Node.js and Express.js. This gives it a competitive advantage since it is able to provide an ideal environment for all user requests to be processed.

The backend is essentially responsible for handling all operations within an application. This includes handling all API routes and all requests coming from the frontend. Additionally, it is responsible for handling all user authentication-related operations. The backend is able to provide secure user authentication through JSON Web Token. This is because it is able to guarantee that only authorized persons access an application.

Some of the major operations performed by the Backend API Server include:

- User registration and login
- Generating JWT token
- Generating OTP and email password recovery
- User role management
- Making API requests
- User input validation

The backend essentially operates on a stateless architecture. This gives it a competitive advantage since it is able to effectively cater to all needs of an application since it is able to process all requests coming from different users.

5. Security and Authentication Layer

Security is a critical component of the system architecture. CommunityLearn implements multiple layers of security to ensure safe access and data protection. JWT-based authentication is used to manage user sessions securely. Tokens are generated during login and verified for each protected request, ensuring that only authenticated users can access system resources.

Additionally, an OTP-based password recovery mechanism is implemented using email services. When a user forgets their password, an OTP is sent to their registered email address. This OTP is verified

before allowing the user to reset their password. This approach enhances account security and prevents unauthorized access.

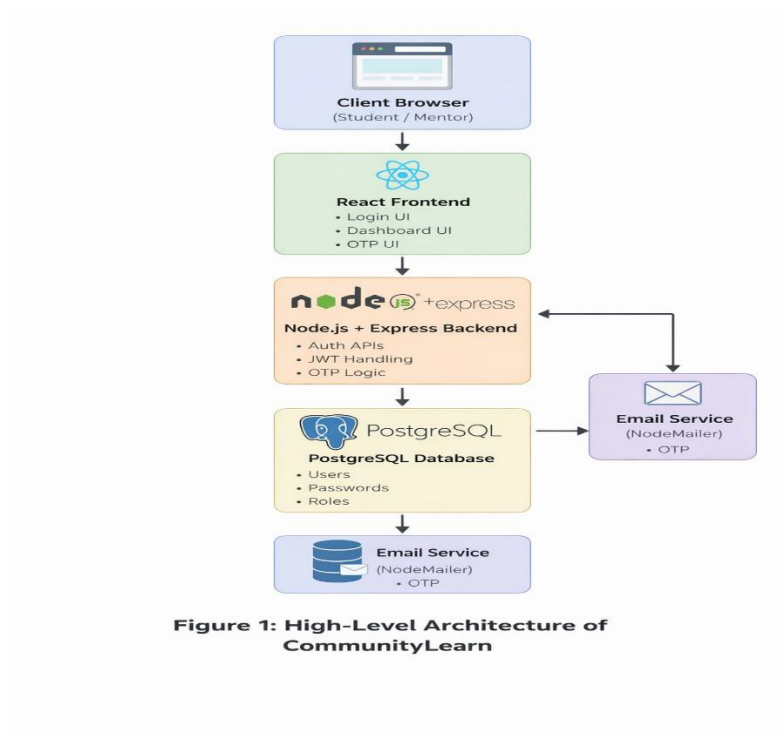
Input validation, secure API communication, and proper error handling are also implemented to maintain system integrity and reliability.

6. Summary

The system architecture of CommunityLearn has created a system that can effectively and efficiently support mentoring activities in a virtual environment. This has been done by dividing the system into three main layers: presentation, application, and data. This has helped in efficient communication.

The addition of secure authentication, role-based dashboards, and a full-stack approach has helped the system in providing a reliable and efficient experience for users. This has been done in the second phase of the implementation and can be further extended in future phases by adding more features.

Figure 1: High-Level Architecture of CommunityLearn



Working Principle

1. Overview of System Workflow

The workflow of the CommunityLearn system is based on the concept of structured interactions between the users (students and mentors), the frontend, the backend, and the database. The workflow of the system ensures the smooth communication and handling of the data with the help of its layered approach. The workflow starts with the interactions between the users and the system through the browser.

Figure 2: Workflow of the User Interaction, Authentication, and Processing within the System

2. User Interaction and Login Flow

The workflow begins when a user accesses the platform through the web frontend. The user can either register a new account or log in using existing credentials. During login, the user enters their email and password, which are sent securely to the backend server.

If the user is new, registration details are submitted and stored in the database after validation. If the user is an existing user, the system verifies credentials by comparing the entered password with the stored encrypted password.

In case the user forgets their password, the system provides a "Forgot Password" option. The user enters their registered email, and an OTP is generated and sent to the email address using the integrated email service. This ensures a secure method for identity verification.

3. Backend Validation and Initial Processing

Once the Backend API Server receives the login and/or registration requests, several processes are carried out:

- Validation of the user's input data, including email and password
- Verification if the user already exists in the database
- Verification of the login credentials by comparing the encrypted password
- Creation of a JWT token after successful authentication

The JWT token is sent back to the client and kept safely in the client-side storage. This token will be used to authenticate future requests. This way, only authorized users can make requests.

For OTP-based password recovery:

- A one-time password (OTP) is created by the backend
- The OTP is stored for a specified time period
- The OTP is sent to the user's email using NodeMailer
- Verification of OTP before password reset

4. Data Processing and Database Interaction

The backend communicates with the PostgreSQL database to store and retrieve user-related data. All sensitive information such as passwords is stored in encrypted form to maintain security.

Data operations include:

- Storing user registration details

- Retrieving user credentials during login
- Updating passwords after successful OTP verification
- Managing user roles (student or mentor)
- Maintaining session-related data

The database ensures consistency and reliability through structured queries and controlled access via the backend server.

5. Dashboard Access and Role-Based Functionality

Once the authentication process is successful, users are redirected to their respective dashboards based on their roles. The system identifies whether the user is a student or a mentor and provides access accordingly.

Students can:

- View their dashboard
- Access mentoring features
- Interact with system functionalities

Mentors can:

- View their dashboard
- Manage sessions and interactions
- Monitor activities related to users

The dashboard interface is simple and easy to use for users to efficiently interact with the system.

6. System Response and User Feedback

The system provides real-time feedback based on user actions. For example:

- Successful login redirects users to dashboards
- Invalid credentials generate error messages
- OTP verification provides confirmation or error alerts
- Password reset confirms successful update

This ensures that users are always informed about the status of their actions, improving overall user experience.

Figure2: Answer Submission and Grading Process



Figure 2: Answer Submission and Grading Process

7. Summary

The working principle of CommunityLearn demonstrates a secure and structured flow of operations, starting from user interaction to backend processing and database management. The integration of JWT-based authentication and OTP-based password recovery ensures system security, while role-based dashboards enhance usability.

This Phase II implementation successfully delivers a functional mentoring platform with reliable authentication, structured workflows, and efficient data handling, making it suitable for real-world educational applications.

Results and Discussion

1. Summary of Phase II Implementation

Phase II of the CommunityLearn project focused on the implementation and development of a fully functional web-based e-mentoring platform. This phase involved converting the conceptual design into a working system using modern web technologies. The major outcomes include the successful integration of frontend, backend, and database components to deliver a secure and user-friendly application.

The key achievements of this phase include:

- Development of a complete full-stack system using React, Node.js, and PostgreSQL
- Implementation of secure user authentication using JSON Web Token (JWT)
- Integration of OTP-based password recovery using NodeMailer
- Creation of role-based dashboards for students and mentors
- Design and implementation of backend API services for handling user requests
- Establishment of structured database schemas for user and system data

This phase demonstrates the practical feasibility of the proposed system and validates the design decisions made in earlier stages.

2. Table 1: Phase II Implementation Milestones

Milestone	Description	Status
Frontend Development	Design and implementation of user interface using React	Completed
Backend Development	API development using Node.js and Express	Completed
Authentication System	JWT-based login and session management	Completed
Password Recovery System	OTP-based email verification using NodeMailer	Completed
Database Integration	PostgreSQL setup and schema implementation	Completed
Dashboard Implementation	Role-based dashboards for students and mentors	Completed

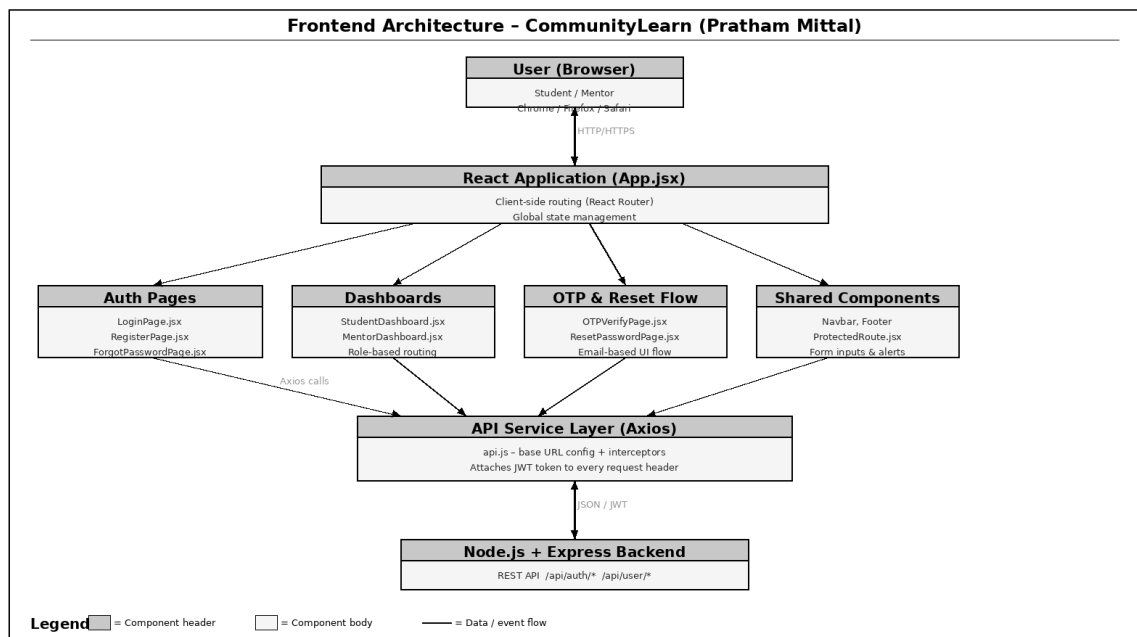
3. Table 2: Achieved Outcomes of Phase II

Outcome	Description
User Authentication System	Secure login and registration using JWT
Frontend Interface	Responsive UI for login, dashboard, and OTP flow
Backend Services	Functional API endpoints for authentication and data handling
Database System	Structured storage of user data and roles
Password Recovery	OTP-based reset system with email verification

Individual Contribution by PRATHAM MITTAL (23BCE10309)

As a core member of the CommunityLearn development team, my primary area of responsibility was Frontend Development and UI Architecture. I designed and built the complete client-side interface that powers the platform's user experience, navigation, authentication flows, and role-based dashboards.

Figure 1: Frontend Architecture Overview



4.1 React Application Setup and Project Structure

I was responsible for initialising and structuring the complete React frontend of the CommunityLearn platform. This included:

- Setting up the React application with a clean folder structure separating pages, components, services, and routing modules.
- Configuring React Router for client-side navigation, establishing routes for all authentication, dashboard, and recovery pages.
- Implementing a ProtectedRoute wrapper component that intercepts navigation attempts by unauthenticated users and redirects them to the login page.
- Managing global application state including the authenticated user's role and JWT token to drive conditional rendering across components.

4.2 Authentication UI – Login and Registration

I designed and implemented the complete authentication interface, which included:

- LoginPage.jsx** — A form accepting email and password, calling POST /api/auth/login and storing the returned JWT token in client-side storage.
- RegisterPage.jsx** — A registration form with fields for name, email, password, and role selection (student or mentor), calling POST /api/auth/register.

- Client-side form validation to check required fields, valid email format, and password length before submitting any request to the backend.
- Displaying appropriate success and error messages based on API responses to guide users through authentication flows.

4.3 OTP-Based Password Recovery Flow

One of the most significant frontend contributions was implementing the complete multi-step OTP password recovery flow:

- `ForgotPasswordPage.jsx` — Accepts the user's registered email and triggers `POST /api/auth/forgot-password` to initiate OTP dispatch.
- `OTPVerifyPage.jsx` — Presents a form for entering the 6-digit OTP received by email, calling `POST /api/auth/verify-otp` for validation.
- `ResetPasswordPage.jsx` — Allows the user to set a new password after successful OTP verification, calling `POST /api/auth/reset-password`.
- Implemented proper state management across pages to carry OTP session context (email, verification status) through the multi-step flow without page reloads.

4.4 Role-Based Dashboard Implementation

I designed and implemented separate dashboards for student and mentor users:

- `StudentDashboard.jsx` — Displays student-specific data including profile information, available mentors, and session details fetched from `GET /api/user/profile`.
- `MentorDashboard.jsx` — Displays mentor-specific data including assigned students, session management options, and interaction history.
- Role detection is performed by decoding the JWT token payload on the client side after login, routing users to the correct dashboard automatically.
- Both dashboards feature a shared `Navbar` component with role-aware navigation links and a logout function that clears the token from storage.

4.5 Axios API Service Layer

- Created a centralised `api.js` module configured with the backend base URL using environment variables.
- Configured Axios request interceptors to automatically attach the JWT token from storage as an `Authorization` header on every outgoing request.
- Configured Axios response interceptors to detect HTTP 401 responses and redirect unauthenticated users back to the login page.
- All API calls in the application route through this centralised layer, ensuring consistent error handling and token management.

4.6 Frontend-Backend Integration Support

- Defined and agreed upon JSON response contracts with the backend developer (Saumil Singhal) to ensure correct data binding in all UI components.
- Resolved CORS-related issues during local development by coordinating with the backend to configure allowed origins and headers correctly.

- Participated in end-to-end integration testing to verify that authentication flows, dashboard data retrieval, and OTP recovery worked correctly across both layers.
- Assisted in configuring the Authorization header format and token prefix (Bearer) to match the backend JWT verification middleware expectations.

CONCLUSION

CommunityLearn, developed in Phase II, successfully demonstrates the implementation of a functional and secure web-based mentoring platform. This phase focused on transforming the conceptual design into a working system by integrating frontend, backend, and database components into a cohesive architecture. The system addresses key challenges in community-based learning environments by providing a structured digital platform for user interaction, secure authentication, and organized access to mentoring resources.

The project highlights the importance of building a reliable and user-friendly system that supports both students and mentors through clearly defined workflows. Features such as user registration, login authentication using JSON Web Tokens (JWT), and OTP-based password recovery using email services ensure that the platform maintains both usability and security. The implementation of role-based dashboards further enhances the user experience by providing customized access and functionality based on user roles.

The modular architecture adopted in CommunityLearn ensures scalability and maintainability, allowing future enhancements without affecting the existing system. The separation of frontend, backend, and database layers enables efficient communication and simplifies system development. Additionally, proper validation, secure data handling, and structured API design contribute to the robustness of the platform.

Although the current system focuses on core functionalities, it establishes a strong foundation for further development. Future work may include the integration of advanced features such as real-time communication between users, analytics dashboards, and intelligent recommendation systems to enhance learning outcomes. These improvements can extend the platform's capabilities and make it more adaptable to diverse educational environments.

In conclusion, CommunityLearn provides a practical and implementable solution for digital mentoring systems. It successfully combines secure authentication, structured workflows, and user-centric design to create an efficient platform that can be further expanded and deployed in real-world community learning scenarios.

REFERENCES

1. Pressman, R. S. "Software Engineering: A Practitioner's Approach." McGraw-Hill, 2014.
2. Sommerville, I. "Software Engineering." Pearson Education, 2016.
3. Kurose, J. F., & Ross, K. W. "Computer Networking: A Top-Down Approach." Pearson, 2017.
4. Fielding, R. "Architectural Styles and the Design of Network-based Software Architectures." Doctoral Dissertation, 2000.
5. Tilkov, S., & Vinoski, S. "Node.js: Using JavaScript to Build High-Performance Network Programs." IEEE Internet Computing, 2010.
6. Haverbeke, M. "Eloquent JavaScript." No Starch Press, 2018.
7. Banks, A., & Porcello, E. "Learning React." O'Reilly Media, 2020.
8. Brown, E. "Web Development with Node and Express." O'Reilly Media, 2019.
9. Flanagan, D. "JavaScript: The Definitive Guide." O'Reilly Media, 2020.
10. Grinberg, M. "Flask Web Development." O'Reilly Media, 2018.
11. Nield, T. "Getting Started with SQL." O'Reilly Media, 2016.
12. Momjian, B. "PostgreSQL: Introduction and Concepts." Addison-Wesley, 2001.
13. Elmasri, R., & Navathe, S. "Fundamentals of Database Systems." Pearson, 2016.
14. Sandhu, R., et al. "Role-Based Access Control Models." IEEE Computer, 1996.
15. Hardt, D. "The OAuth 2.0 Authorization Framework." IETF, 2012.
16. Jones, M., et al. "JSON Web Token (JWT)." IETF RFC 7519, 2015.
17. OWASP Foundation. "OWASP Top 10 Web Application Security Risks." 2021.
18. Stallings, W. "Cryptography and Network Security." Pearson, 2017.
19. Rescorla, E. "HTTP Over TLS." IETF RFC 2818, 2000.
20. Richardson, L., & Ruby, S. "RESTful Web Services." O'Reilly Media, 2007.
21. Newman, S. "Building Microservices." O'Reilly Media, 2015.
22. Bass, L., Clements, P., & Kazman, R. "Software Architecture in Practice." Addison-Wesley, 2012.
23. Gamma, E., et al. "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley, 1994.

24. Pautasso, C., et al. "RESTful Web Services vs. Big Web Services." WWW Conference, 2008.
25. Deitel, P., & Deitel, H. "Internet & World Wide Web: How to Program." Pearson, 2012.
26. React Documentation. "React Official Documentation." <https://react.dev>
27. Node.js Foundation. "Node.js Official Documentation." <https://nodejs.org>
28. PostgreSQL Global Development Group. "PostgreSQL Documentation." <https://www.postgresql.org/docs/>
29. Mozilla Developer Network (MDN). "Web Docs for HTML, CSS, and JavaScript." <https://developer.mozilla.org>
30. W3C. "Web Standards and Guidelines." <https://www.w3.org>

PUBLICATION / CONFERENCE / PATENT

The present work, which is titled "CommunityLearn: Web-Based Student-Mentor Platform with Secure Authentication," has not been submitted or published in any journal, conference, or patent at the time of the submission of the report.

The project is currently developed as a part of the EPICS Phase II academic requirements at VIT Bhopal University, and the future work can be to enhance the project and submit it to relevant conferences or journals in the domain of web-based education systems and e-learning platforms.

BIODATA

PRATHAM MITTAL (23BCE10309)



Pratham Mittal is a student at VIT Bhopal University pursuing a Bachelor of Technology in Computer Science and Engineering (Core). Scalable application design, backend architecture, and full-stack web development are among his areas of interest. He has worked with relational databases, structured modular software systems, and developed RESTful APIs. Pratham concentrated on system architecture, backend workflow planning, and integration points between AI modules and core services as part of the EduConnect project. His goal is to work as a software engineer on high-availability, large-scale systems.

SAUMIL SINGHAL (23BAI10872)



At VIT Bhopal University, Saumil Singhal is pursuing a B.Tech in Computer Science and Engineering with a focus on Artificial Intelligence. He has a strong interest in natural language processing, deep learning, and machine learning. He has experience with information retrieval methods, transformer-based language models, and supervised learning pipelines. Saumil contributed to the EduConnect project by investigating automated short-answer grading techniques, creating the AI Tutor Assistant concept, and writing report sections pertaining to AI. His goal is to work in intelligent systems and applied AI.

SHIKHAR GUPTA (23BSA10035)



Shikhar Gupta is a student at VIT Bhopal University studying computer science with a focus on cloud technology. Cloud-native application development, distributed computing, containerization, and DevOps techniques are among his areas of academic interest. He has experience creating microservice-based architectures and deploying services on cloud platforms. Shikhar helped EduConnect plan the infrastructure needed to host AI services, as well as the deployment architecture and scalability considerations. His goal is to work in contemporary software environments as a cloud and DevOps engineer.

PRAVEEN SINGH (23BCY10191)



At VIT Bhopal University, Praveen Singh is pursuing a B.Tech in Computer Science and Engineering with a focus on cybersecurity. He is interested in secure system design, network security, and vulnerability assessment. He has researched risk analysis models, encryption methods, and authentication systems. Praveen made contributions to the EduConnect project by determining the security and privacy needs, creating role-based access control, and thinking about safe data handling techniques. He intends to pursue a career in secure software engineering and cybersecurity.

AMAN KUMAR GUPTA (23BCY10119)



Aman Kumar Gupta is a student in VIT Bhopal University's Computer Science and Engineering program with a focus on cybersecurity. He is interested in system hardening, secure coding techniques, and information security. He has been researching security techniques and tools for data and application protection. Aman supported the documentation of security-related design choices, reviewed potential vulnerabilities, and helped model secure data flows in the EduConnect project. His career goals are to work in the fields of secure application development and cyber defense.

SAMBHAV CHHAYALA (23BCE10380)



Sambhav Chhayala is a student at VIT Bhopal University studying Computer Science and Engineering (Core). He is interested in data-driven applications, problem solving, and software development. He has experience creating user interfaces, organizing application logic, and recording technical workflows. Sambhav helped with requirements analysis, the design of learner and tutor interaction flows, and the arrangement of literature sources and report content for EduConnect. His goal is to work in software engineering, specializing in intelligent, user-centered systems.